

Experiment No. 4

Frequency Domain Image Filtering using Fourier Transform

Aim: To implement image filtering in the frequency domain using the Fourier Transform and analyze Low-Pass and High-Pass filtering for noise removal, image enhancement, and feature preservation using Python and OpenCV.

Theory

The Fourier Transform converts an image from the **spatial domain** into the **frequency domain**. Low-frequency components mainly represent smooth and slowly varying intensity information, while high-frequency components contain edges, fine details, and much of the noise.

- **DFT:** Converts the image to the frequency domain.
- **FFT Shift:** Moves the zero-frequency component to the center.
- **Low-Pass Filter (LPF):** Keeps low frequencies and suppresses high frequencies.
- **High-Pass Filter (HPF):** Keeps high frequencies and suppresses low frequencies.
- **IDFT:** Converts the filtered frequency representation back to the spatial domain.

```
# Step 1: Import required libraries
import cv2
import numpy as np
import matplotlib.pyplot as plt
from google.colab import files

print("Libraries imported successfully.")

Libraries imported successfully.

# Step 2: Upload and load a grayscale image
uploaded = files.upload()

file_name = next(iter(uploaded))
img = cv2.imread(file_name, cv2.IMREAD_GRAYSCALE)

if img is None:
    raise ValueError("Image could not be loaded. Please upload a valid image.")

print("Image:", file_name)
print("Image size:", img.shape)

plt.figure(figsize=(6, 5))
plt.imshow(img, cmap="gray")
```

```
plt.title("Original Grayscale Image")
plt.axis("off")
plt.show()
```

<IPython.core.display.HTML object>

Saving 9011064.webp to 9011064.webp
Image: 9011064.webp
Image size: (2160, 3840)

Original Grayscale Image



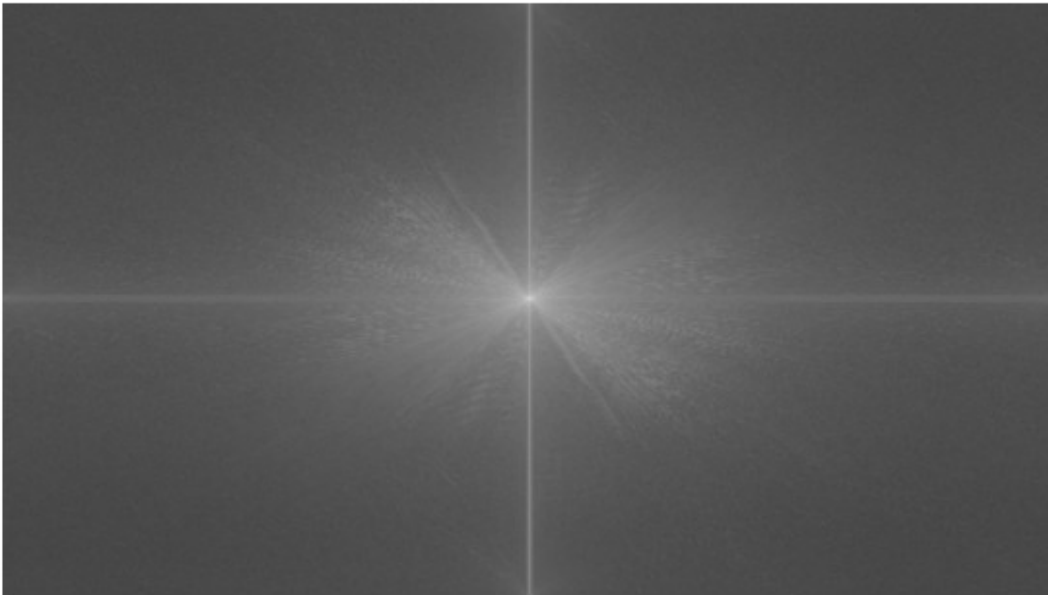
```
# Step 3: Compute DFT and shift the zero-frequency component
img_float = np.float32(img)

dft = cv2.dft(img_float, flags=cv2.DFT_COMPLEX_OUTPUT)
dft_shift = np.fft.fftshift(dft)

# Calculate magnitude spectrum
magnitude = cv2.magnitude(dft_shift[:, :, 0], dft_shift[:, :, 1])
magnitude_spectrum = 20 * np.log(magnitude + 1)

plt.figure(figsize=(7, 5))
plt.imshow(magnitude_spectrum, cmap="gray")
plt.title("Magnitude Spectrum")
plt.axis("off")
plt.show()
```

Magnitude Spectrum



Step 4: Low-Pass Frequency Filter

A circular mask is created around the center of the shifted spectrum. Frequencies inside the radius are retained and high-frequency components outside the radius are suppressed.

```
# Step 5: Apply Low-Pass Filter
rows, cols = img.shape
crow, ccol = rows // 2, cols // 2

# Radius controls the amount of smoothing
radius = min(rows, cols) // 8

y, x = np.ogrid[:rows, :cols]
distance = np.sqrt((x - ccol) ** 2 + (y - crow) ** 2)

low_pass_mask = (distance <= radius).astype(np.float32)

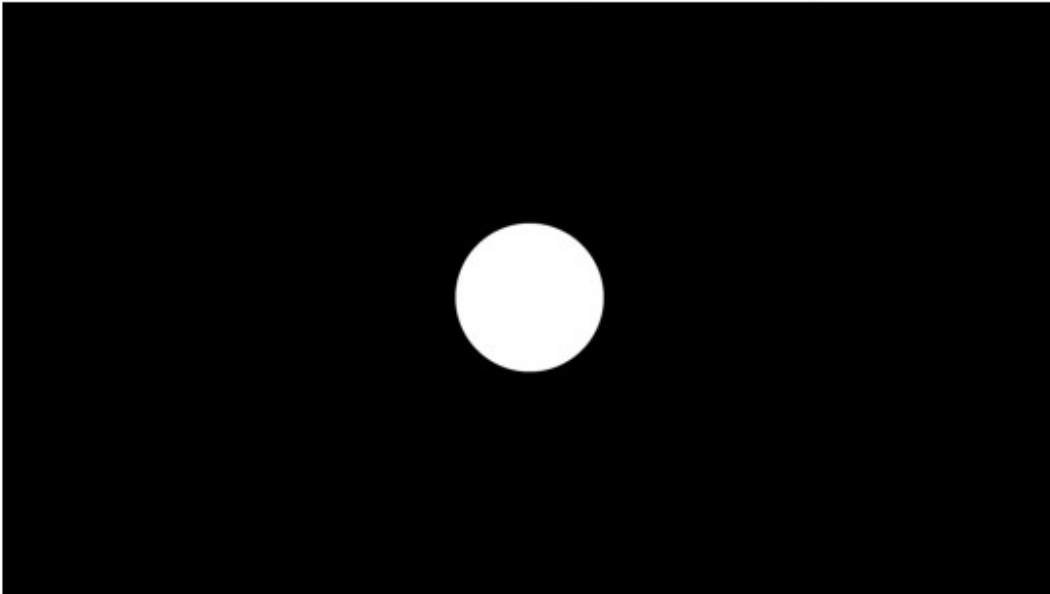
# Apply mask to both DFT channels
low_pass_dft = dft_shift * low_pass_mask[:, :, np.newaxis]

# Shift back before inverse DFT
low_pass_ishift = np.fft.ifftshift(low_pass_dft)

low_pass_result = cv2.idft(low_pass_ishift)
low_pass_result = cv2.magnitude(
    low_pass_result[:, :, 0],
    low_pass_result[:, :, 1]
)
```

```
low_pass_result = cv2.normalize(  
    low_pass_result, None, 0, 255, cv2.NORM_MINMAX  
) .astype(np.uint8)  
  
plt.figure(figsize=(7, 5))  
plt.imshow(low_pass_mask, cmap="gray")  
plt.title(f"Low-Pass Filter Mask (Radius = {radius})")  
plt.axis("off")  
plt.show()  
  
plt.figure(figsize=(7, 5))  
plt.imshow(low_pass_result, cmap="gray")  
plt.title("Low-Pass Filtered Image")  
plt.axis("off")  
plt.show()
```

Low-Pass Filter Mask (Radius = 270)



Low-Pass Filtered Image



Step 6: High-Pass Frequency Filter

A high-pass mask is obtained by removing the central low-frequency region from the frequency spectrum. This retains high-frequency information such as edges and fine details.

```
# Step 7: Apply High-Pass Filter
high_pass_mask = 1.0 - low_pass_mask

# Apply mask to both DFT channels
high_pass_dft = dft_shift * high_pass_mask[:, :, np.newaxis]

# Shift back before inverse DFT
high_pass_ishift = np.fft.ifftshift(high_pass_dft)

high_pass_result = cv2.idft(high_pass_ishift)
high_pass_result = cv2.magnitude(
    high_pass_result[:, :, 0],
    high_pass_result[:, :, 1]
)

high_pass_result = cv2.normalize(
    high_pass_result, None, 0, 255, cv2.NORM_MINMAX
).astype(np.uint8)

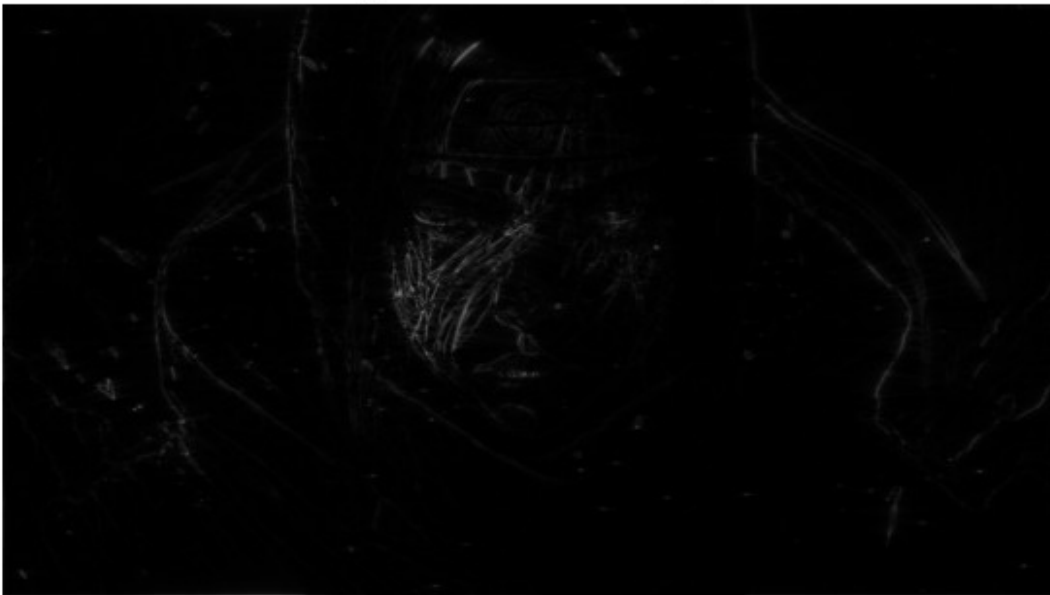
plt.figure(figsize=(7, 5))
plt.imshow(high_pass_mask, cmap="gray")
plt.title(f"High-Pass Filter Mask (Radius = {radius})")
plt.axis("off")
plt.show()
```

```
plt.figure(figsize=(7, 5))  
plt.imshow(high_pass_result, cmap="gray")  
plt.title("High-Pass Filtered Image")  
plt.axis("off")  
plt.show()
```

High-Pass Filter Mask (Radius = 270)



High-Pass Filtered Image



Step 8: Compare all results

```
# Step 8: Final comparison
plt.figure(figsize=(16, 4))

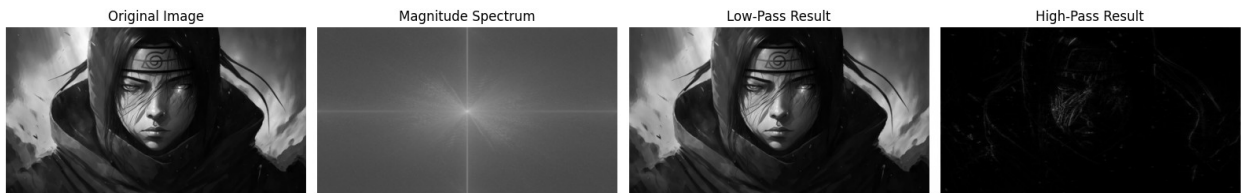
plt.subplot(1, 4, 1)
plt.imshow(img, cmap="gray")
plt.title("Original Image")
plt.axis("off")

plt.subplot(1, 4, 2)
plt.imshow(magnitude_spectrum, cmap="gray")
plt.title("Magnitude Spectrum")
plt.axis("off")

plt.subplot(1, 4, 3)
plt.imshow(low_pass_result, cmap="gray")
plt.title("Low-Pass Result")
plt.axis("off")

plt.subplot(1, 4, 4)
plt.imshow(high_pass_result, cmap="gray")
plt.title("High-Pass Result")
plt.axis("off")

plt.tight_layout()
plt.show()
```



Observations

1. The magnitude spectrum shows the distribution of frequency components of the image.
2. The center of the shifted spectrum contains low-frequency components, while regions farther from the center represent higher frequencies.
3. The Low-Pass Filter suppresses high-frequency components, so the output becomes smoother and fine noise/details are reduced.
4. The High-Pass Filter suppresses low-frequency components and emphasizes edges, fine structures, and rapid intensity changes.
5. Increasing the LPF radius preserves more image details; decreasing it produces stronger smoothing.
6. A simple ideal HPF produces an edge/detail image. In practical applications, high-pass information can be combined with the original image for sharpening.

Result

Frequency-domain image filtering was successfully implemented using DFT, frequency shifting, Low-Pass and High-Pass filters, and inverse DFT. Low-pass filtering produced smoothing/noise reduction, while high-pass filtering highlighted edges and fine details.

Questions and Answers

1. What is the Fourier Transform? Why is it important in digital image processing?

Answer: Fourier Transform is a mathematical technique that converts an image from the spatial domain into the frequency domain. It represents the image using different frequency components. It is important for filtering, noise removal, enhancement, restoration, and edge analysis.

2. Differentiate between the spatial domain and the frequency domain.

Answer: In the spatial domain, processing is performed directly on pixel values. In the frequency domain, the image is represented by frequency components obtained using a transform such as Fourier Transform. Spatial filtering is usually local, while frequency filtering modifies selected frequency ranges.

3. What is the significance of the Discrete Fourier Transform (DFT) in image processing?

Answer: DFT converts a discrete digital image from the spatial domain into its frequency representation. It allows low- and high-frequency components to be analyzed and selectively filtered. It is useful for enhancement, restoration, noise reduction, and feature analysis.

4. Explain the purpose of shifting the zero-frequency component to the center of the frequency spectrum.

Answer: Normally, the zero-frequency component is located at the corners of the Fourier spectrum. Shifting it to the center makes the spectrum easier to visualize and makes circular frequency-domain filter design convenient. `np.fft.fftshift()` is used for this purpose.

5. Compare Low-Pass Frequency Filters and High-Pass Frequency Filters with suitable applications.

Answer: A Low-Pass Filter allows low frequencies and suppresses high frequencies. It is used for smoothing and noise reduction. A High-Pass Filter allows high frequencies and suppresses low frequencies. It is used for edge detection, detail enhancement, and feature extraction.

6. What is the role of the Inverse Fourier Transform (IDFT) in image reconstruction?

Answer: IDFT converts the filtered frequency-domain representation back into the spatial domain. It reconstructs the image after selected frequency components have been modified or removed.

7. Why is frequency domain filtering preferred for certain image enhancement tasks?

Answer: Frequency-domain filtering provides direct control over different frequency components. It is useful for removing periodic noise, smoothing, sharpening, restoration, and suppressing or emphasizing selected frequency ranges.

8. Mention four real-world applications where Fourier Transform is used in computer vision and image analysis.

Answer:

1. Medical image enhancement and analysis.
2. Satellite and remote-sensing image processing.
3. Image restoration and periodic-noise removal.
4. Biometric and feature-analysis systems.

9. Compare frequency domain filtering with spatial domain filtering based on computational efficiency and practical applications.

Answer: Spatial filtering directly operates on pixels and is simple for small local kernels such as mean, median, and Sobel filters. Frequency-domain filtering can be efficient for large filters because convolution can be performed using multiplication in the frequency domain, especially with FFT-based methods. Frequency-domain methods are useful for large-scale smoothing, sharpening, restoration, and periodic-noise removal.

10. How does frequency domain filtering improve the performance of image restoration and feature extraction techniques?

Answer: It separates image information according to frequency, allowing unwanted components such as noise to be suppressed while retaining useful components. Low-pass filtering can reduce noise, while high-pass filtering emphasizes edges and fine details, improving feature extraction.

Conclusion

The experiment successfully demonstrates frequency-domain image filtering using the Fourier Transform. The Low-Pass Filter produces a smoother image by reducing high-frequency components, whereas the High-Pass Filter emphasizes edges and fine details by suppressing

low-frequency components. Fourier-based filtering is therefore useful for image enhancement, restoration, noise reduction, and feature extraction.